

Informed Search (Heuristic search techniques)

Dr. Manas Ranjan Patra

Uninformed vs Informed Search

- In uninformed search, one can only know whether it is a goal state or not. But, cannot say whether a non-goal state is better than another non-goal state. Therefore, it's a kind of “**blind search**” as we continue to search with a hope that the goal state can be reached, if it exists.
- But, in case of informed search one can evaluate how good/desirable a state is compared to other states with the help of additional information specific to the problem.

Heuristic Search:

Greek word: “Heuriskein” -To discover

Archimedes: “Eureka”- I have found

Observation: “Human being are not optimizers rather satisfiers”.

We don't always look for the “Best solution” but yet satisfied with a “good approximated solution”.

Ex:- i) Looking for a seat in a crowded bus.

ii) Looking for a parking slot.

Choice of Heuristic (Rule of thumb) depends on the problem characteristics.

Design of Heuristics/Heuristic Function(HF) :

- A HF is a function which maps from problem state descriptions to measures of desirability.
- A HF is designed by assigning appropriate weightage to the significant aspects of a problem state.

Issues:

- How to formulate an effective HF?
- What weightage to be given to which parameter?
- Deciding on whether to maximize or minimize the HF value, which depends on the nature of the problem.

Aim:

The purpose of a HF is to guide the search process in the most profitable direction in the presence of multiple alternatives.

Observation:

There is a trade-off between the cost of evaluating a HF & the savings in search time that the HF provides.

Heuristic Search Techniques

Generate and Test(GAT):

Algorithm:

1. Generate a possible solution.
2. Test if it is the desired solution.
3. If solution found then Quit else goto step1.

Remark:

- If solutions are generated systematically, a solution can be obtained(if exists),but takes longtime.
- If solutions are generated randomly, no guarantee for a solution but sometimes a solution can be found in less time.

Ex: “**British Museum Algorithm**”

(Searching of an item in an Unordered List)

Possible Approach:

Search systematically but ignore a path if found unlikely to lead to a solution.

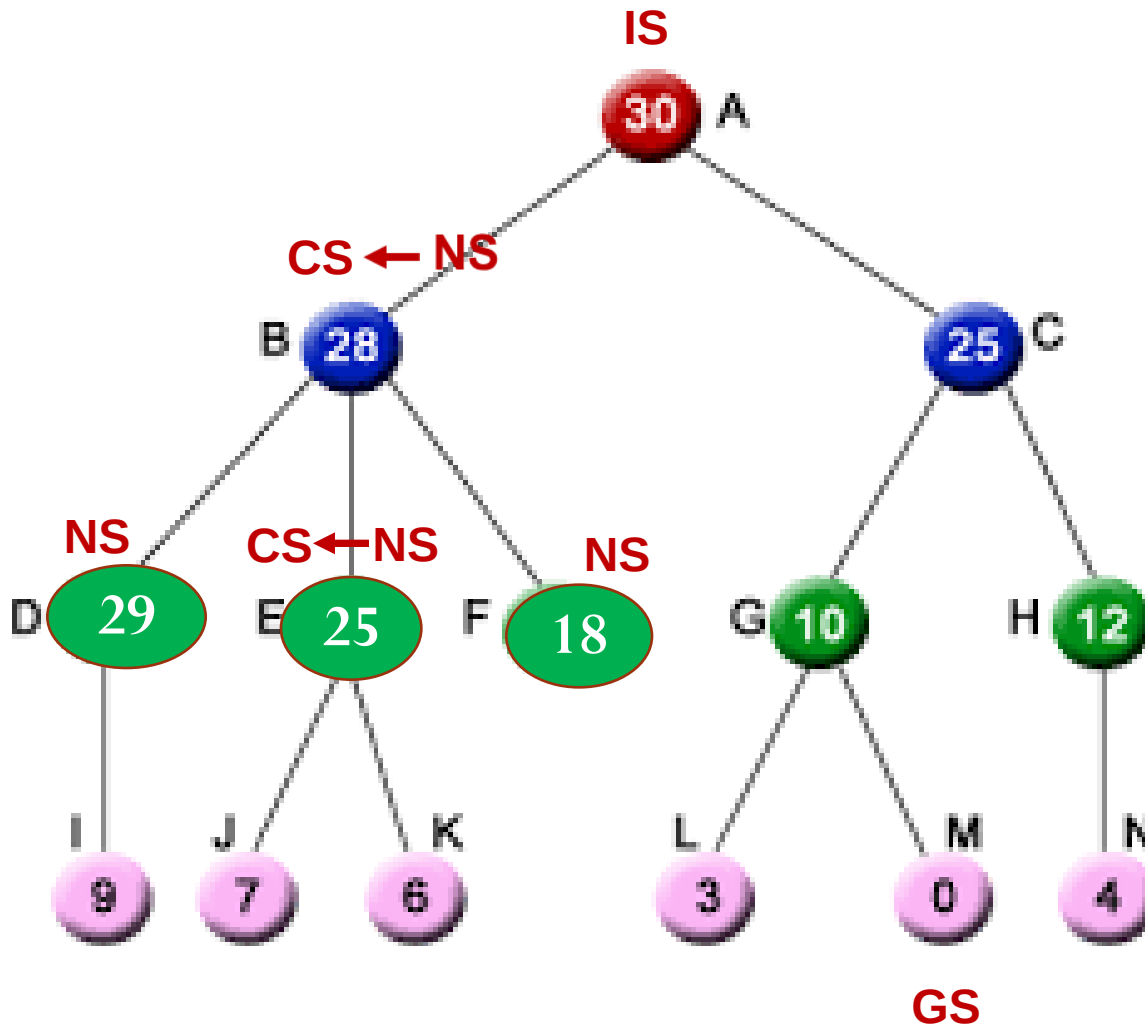
A path can be evaluated by a suitable “Heuristic Function”.

Generate-And-Test” may be modified to “PLAN-GENERATE-TEST”, while planning use some “Constraint-satisfaction techniques”.

HILL Climbing

Algorithm: Simple Hill Climbing

1. Evaluate the Initial State(**IS**):
 - if it is a Goal State(**GS**) then Return Solution & Quit.
 - else consider the **IS** as the current state & proceed further.
 2. Loop until a solution is found or there are no new operators to be applied in the current State(**CS**).
 - A) Select an operator(**op**) that has not yet been applied to the **CS**.
Apply it to generate a New State(**NS**).
 - B) Evaluate the **NS**
 - i) If **NS** is a **GS**, then return Solution & Quit.
 - ii) If not but **NS** better than the **CS** then consider that as the **CS** ($CS \leftarrow NS$) & proceed
 - iii) If **NS** is not better than **CS** then continue in the loop by selecting the next applicable **operator** for **CS**.
- [End of loop]
- (Trying to locate a place in a big unfamiliar city w.r.t a tall building.)



Steepest-Ascent Hill Climbing

This approach evaluates all alternatives at a **CS** & selects the best one as the **NS**. It is also called “**Gradient Search**”.

Algorithm:

1. Evaluate the **IS**:

If it is a **GS** then Return Solution & Quit.

else proceed with the **IS** as the **CS**.

2. Loop until a solution is found or a complete iteration produces no change to the **CS**.

A) Apply a valid operator to **CS** & generate a **NS** and set it to **succ** ($\text{succ} \leftarrow \text{NS}$)

B) For each applicable op to **CS** do:

i) Apply op to **CS** & generate a **NS**.

ii) Evaluate the **NS**

If it is a **GS**, then Return Solution & Quit.

Else compare it with **succ**.

If **NS** is better than **succ** then set **succ** to **NS**

[i.e. **succ** ← **NS**]

Else leave **succ** unchanged.

C) If **succ** is better than the **CS** then

set **CS** to **succ** [i.e. **CS** ← **succ**]

i.e. move to the next best state.

[End of loop]

Drawbacks of Hill Climbing Algorithm:

HC may fail to find a solution when the algorithm terminates by reaching a state from where no better states can be generated.

This can happen when the HC algorithm reaches either a “**Local maximum**” or “**Plateau**” or a “**Ridge**”.

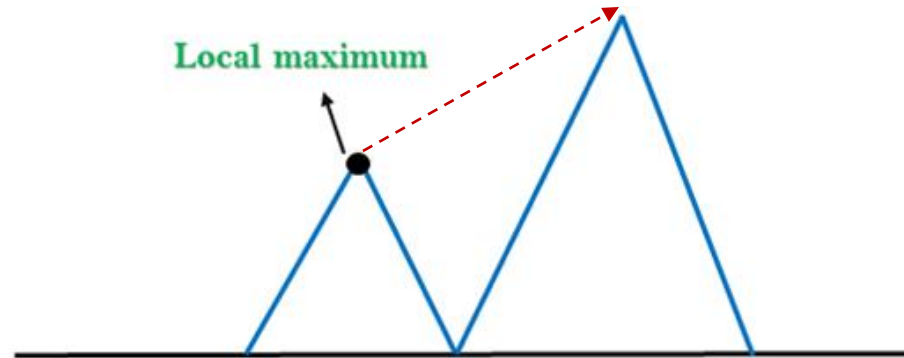
Observation:

The HC algorithm suffers from 3 drawbacks as it uses only local info, rather it must use global info.

Local Maximum:

“Foot Hills”

It is a state in the search space which is better than all its neighbors but is not better than some state further away.

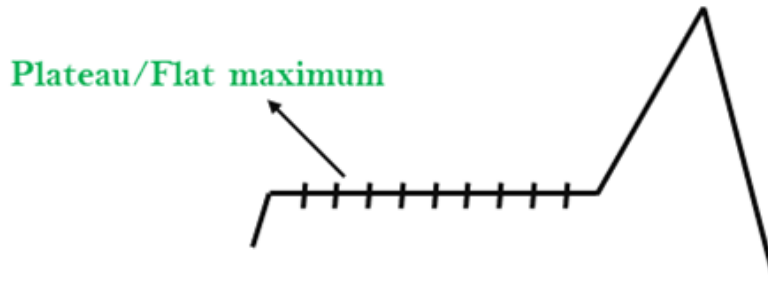


Solution:

Maintain a list of promising paths so that the algorithm can backtrack to an ancestor in the search space and explore other alternate paths.

“Plateau”

It is a flat area in the search space in which all the neighboring states have the same HF value.



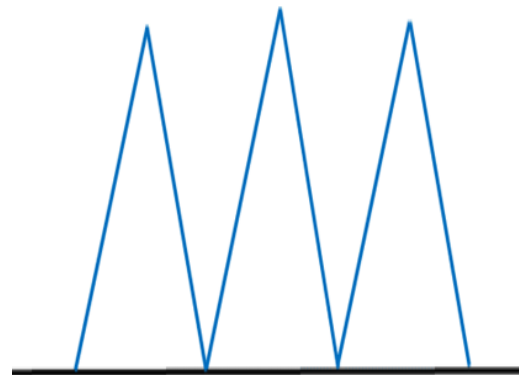
i.e. It is not possible to find the best direction by making local comparisons.

Solution:

- Take big steps, that is expand few generation ahead to move to a different section of the search space.
- Randomly select a state which is far away from the current state in order to move to a non-plateau region.

Ridge: (a type of “Foot Hill”)

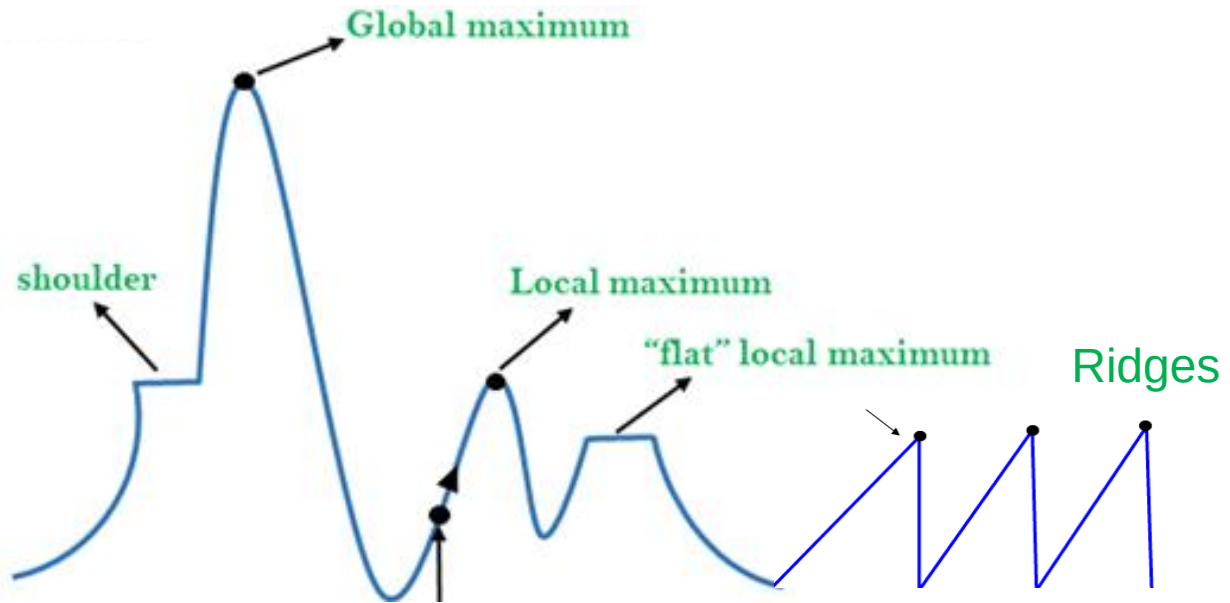
A ridge is a special form of the local maximum. It is an area in the search space which is higher than its surrounding but itself has slopes, and cannot be reached in a single move as no such operator is available.



Solution:

Apply several operators before doing the evaluation. Move in different directions until you come out of the ridge.

Different regions in the state space



Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum is the best possible state in the state space

Flat local maximum/Plateau: It is a flat space in the state space where all the neighbor states of a current state have the same value.

Shoulder: It is a plateau region which has an uphill edge.

Ridges: Sharp slopes

Local maxima, Plateau, Ridge Problems

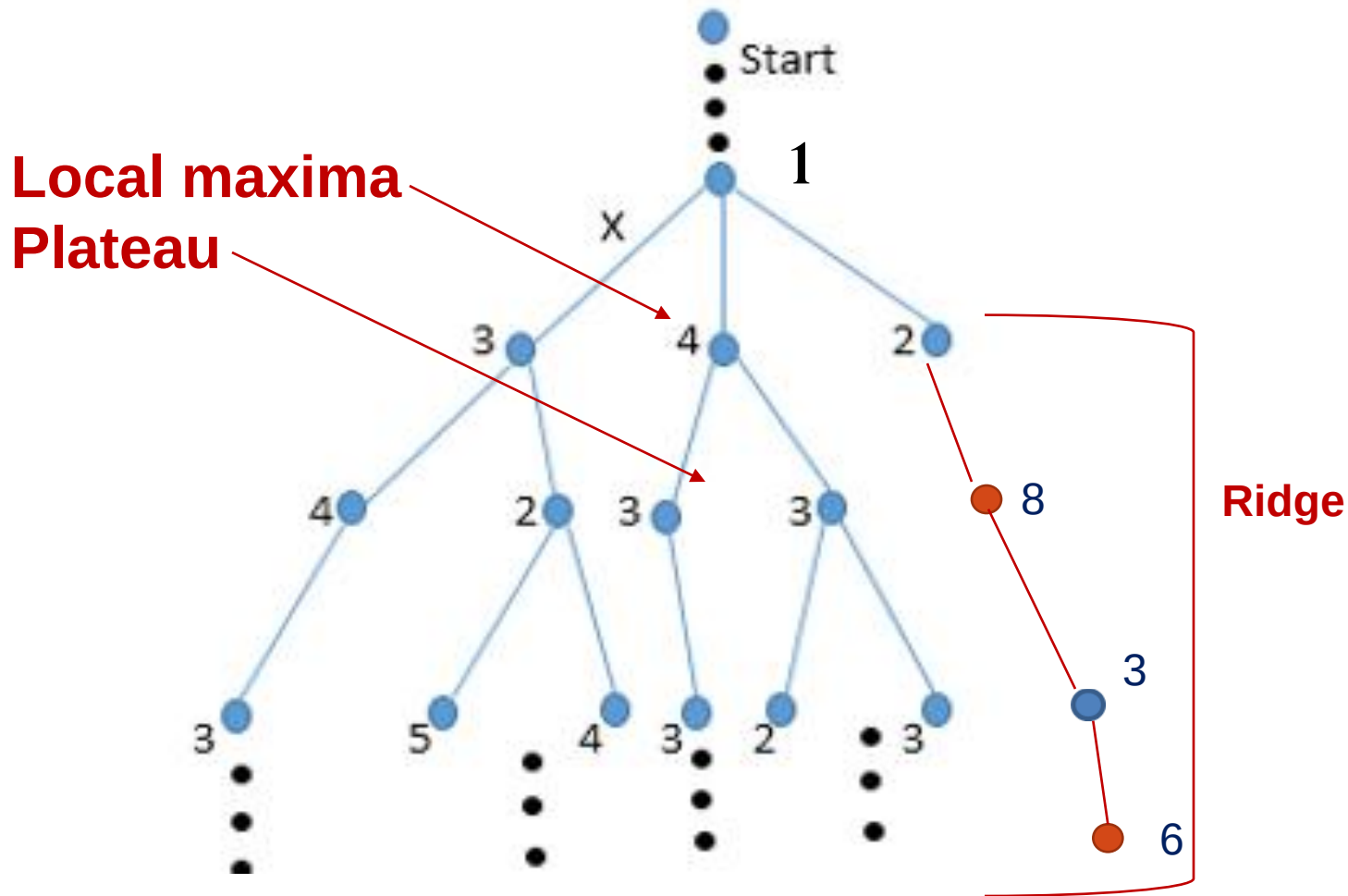
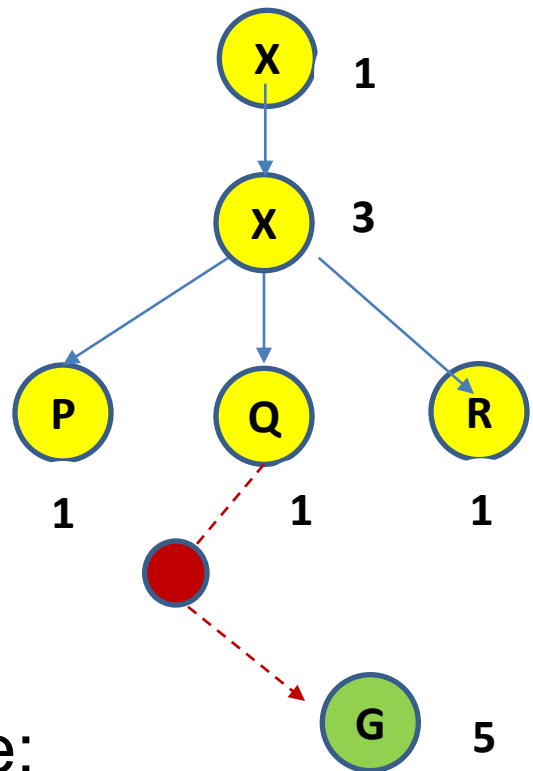
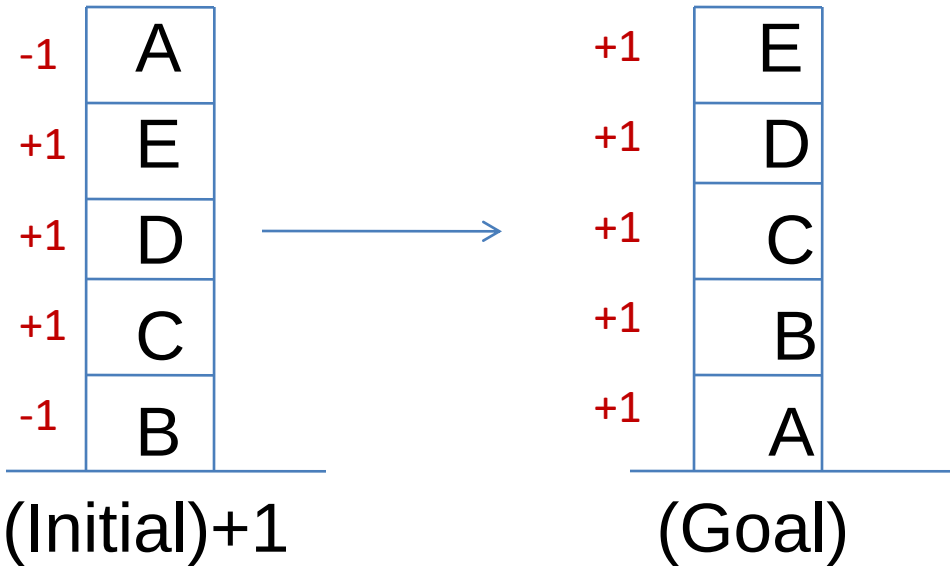
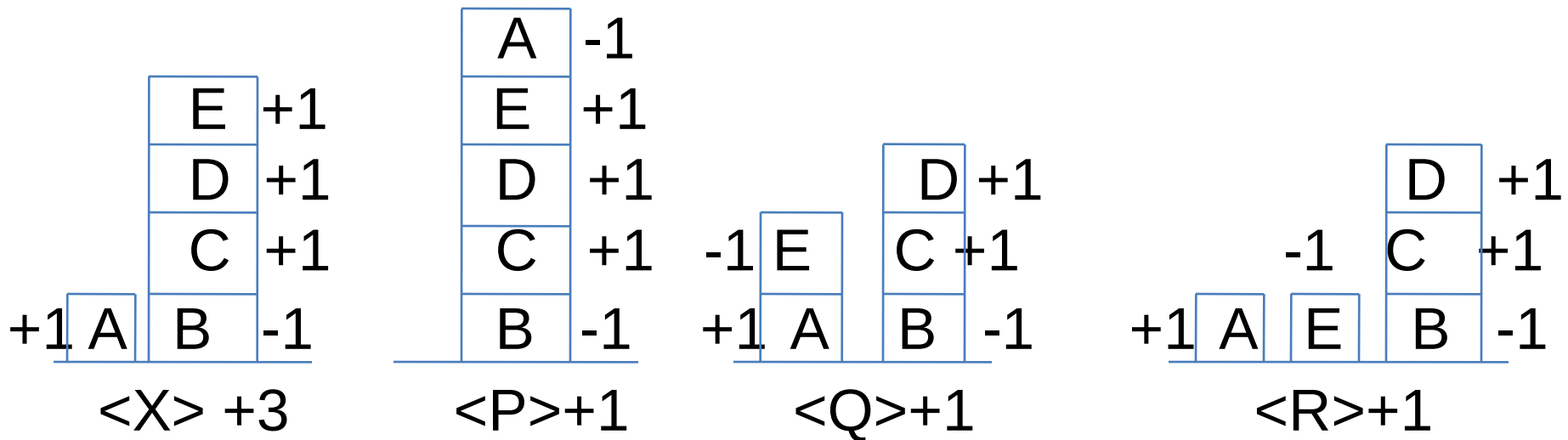


Illustration:



3 possible moves from the NS 'X' are:



Heuristic function used:

1. Assign +1 for every block correctly placed w.r.t. the Goal state.
2. Assign -1 for every block misplaced under a specified block.

Conclusion:

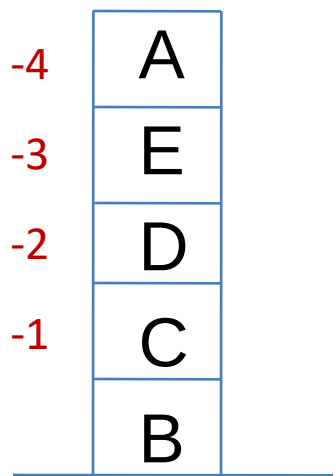
If one uses only local information there is possibility of getting stuck in the state space while moving from an Initial state to a Goal state.

Modified Heuristic

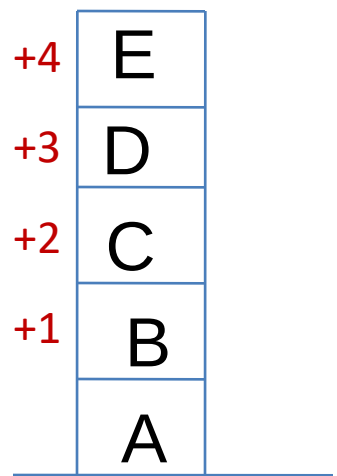
Strategy:

- Incorrect structures are useless, hence ignore them.
 - Correct structures are useful, hence build upon them.
-
1. Assign +1 for each of the blocks correctly placed in the support structure w.r.t. the Goal state.
 2. Assign -1 for each of the blocks misplaced in the support structure under a specified block.

Illustration:

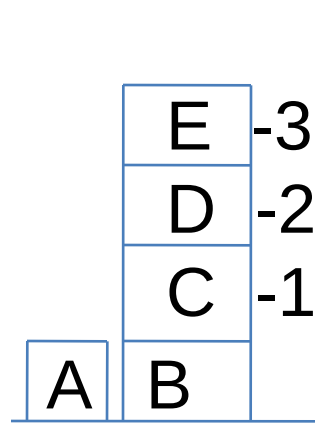


(Initial)-10

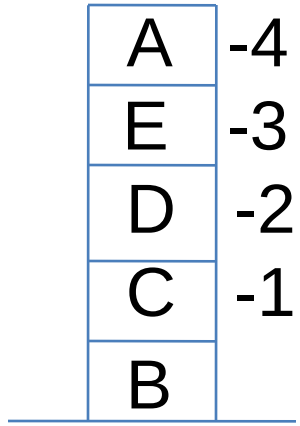


(Goal)

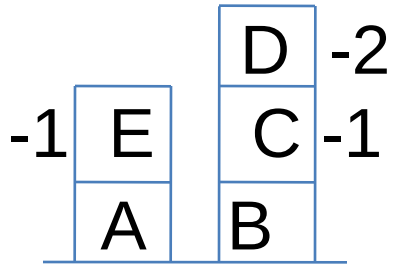
3 possible moves from the NS 'X' are:



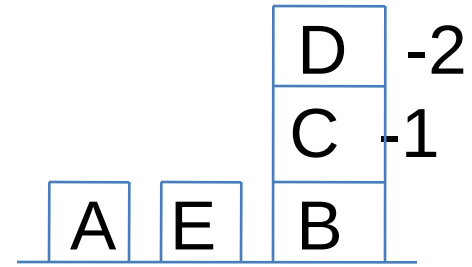
<X> - 6



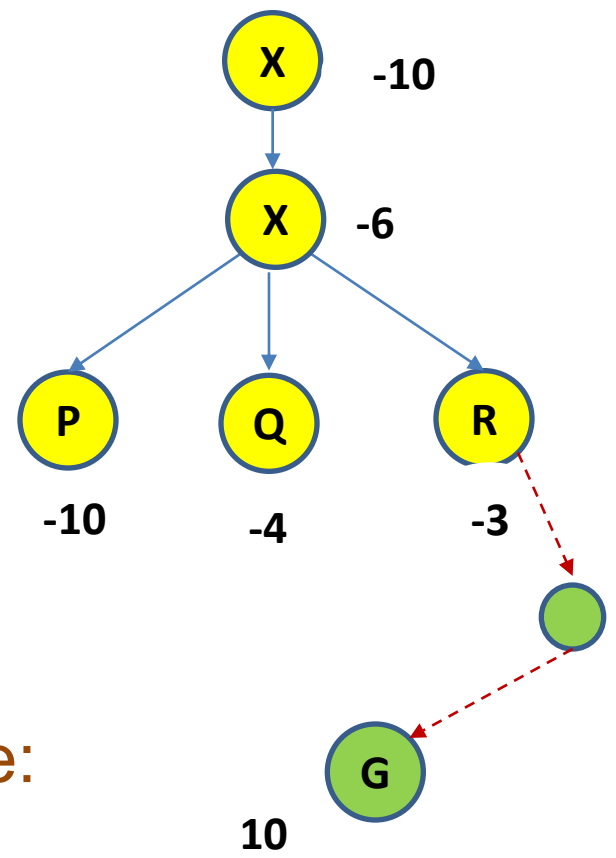
<p>-10



<Q>-4



<R>-3



Best First Search

This combines good features of both DFS & BFS.
DFS doesn't require all branches to be expanded &
BFS doesn't get trapped at dead-ends.

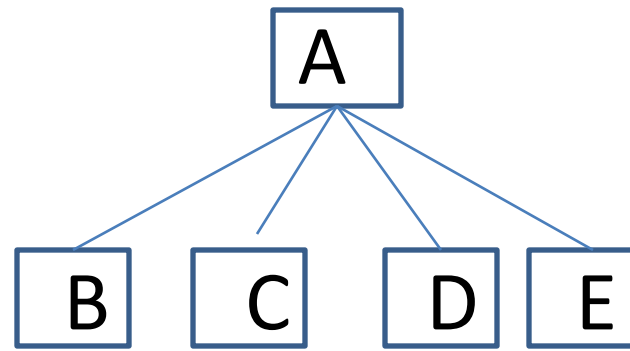
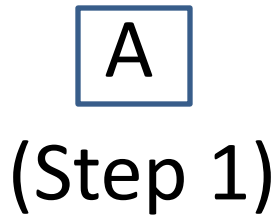
Approach

Follow a single path but switch over to a different path if it appears to be more promising at any stage.

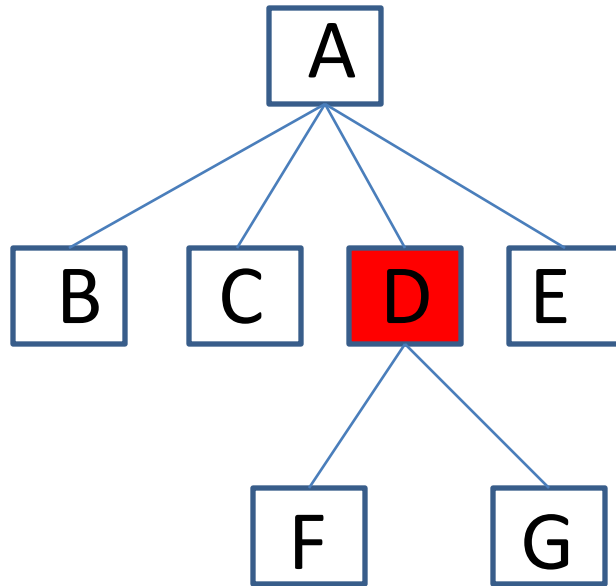
Best-First Strategy:

The most promising node (state) among all the nodes generated so far is selected at each step.

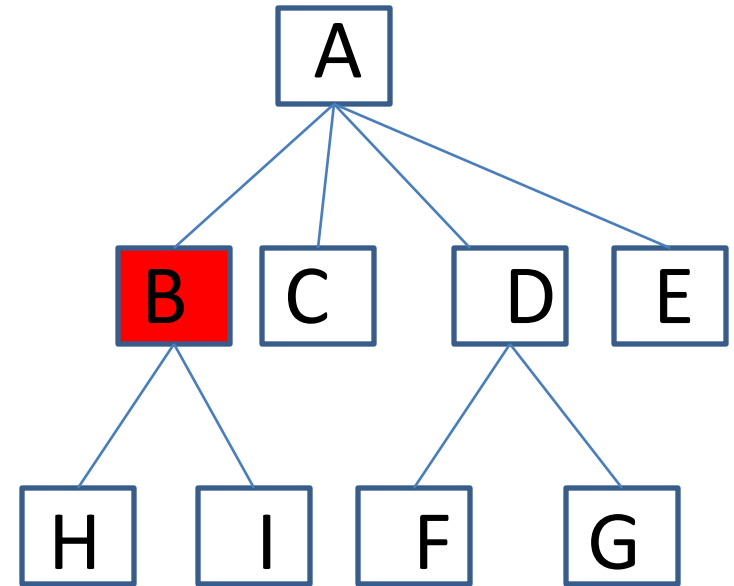
- A promising node is selected by applying a suitable HF to each competing nodes.
- Less promising nodes are added to a list for future evaluation (in case the selected ones prove to be less promising at a later stage).
- Generation of new nodes continue by selecting the most promising one at each step.



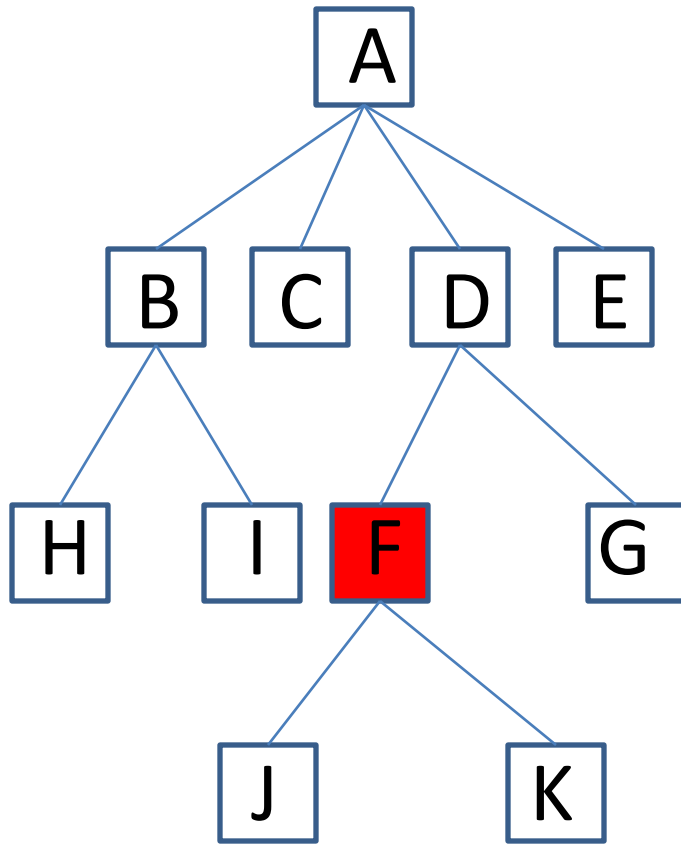
(Step 2)



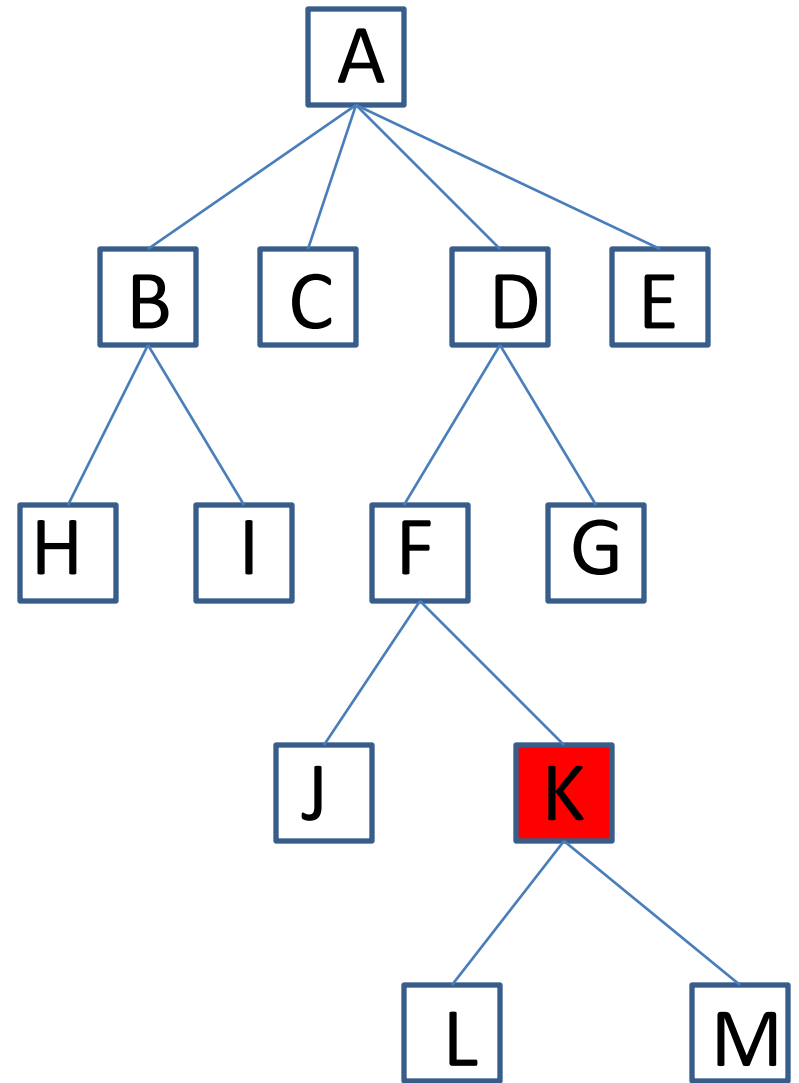
(Step 3)



(Step 4)



(Step 5)



(Step 6)

----- and so on till solution is obtained.

Evaluation of HF:

Let,

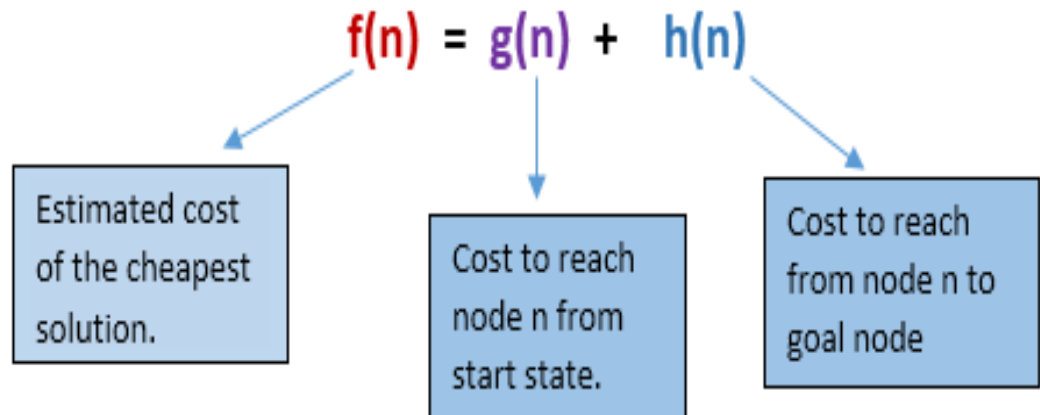
f : Actual HF value which can give the true evaluation of a node.

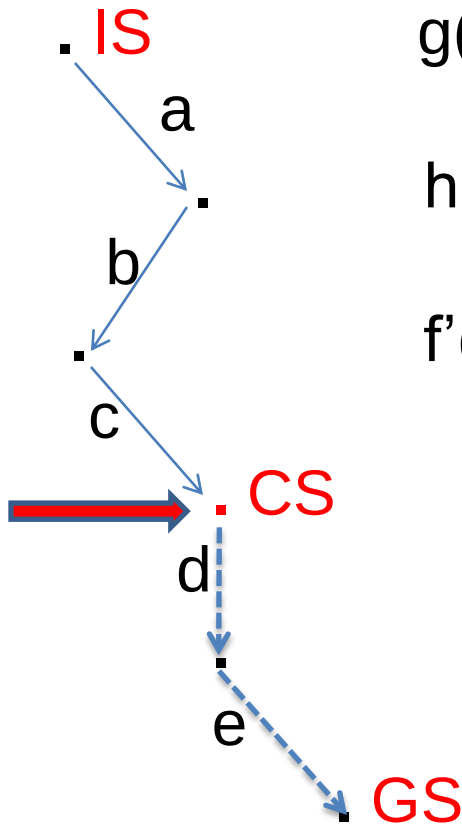
f' : An approximation to f (f' is the HF value for all practical purposes).

G : function which gives the actual cost incurred in reaching CS from IS. $\text{cost}(\text{IS} \rightarrow \text{CS})$

h' : function which gives an estimate of the additional cost to reach GS from CS. $\text{cost}(\text{CS} \rightarrow \text{GS})$.

$$f' = g + h'$$





$g(\text{CS}) = a + b + c$ (weights indicate the cost factor)

$h'(\text{CS}) = d + e$

$f'(\text{CS}) = g + h'$
 $= (a + b + c) + (d + e)$

Thus, f' gives an estimate of the cost of moving from the IS to the GS through the current node under consideration.
 f' should be minimal(min/max) in order that the selected path is the best path.

IS

1	2	3
	4	6
7	5	8

$g=0, h=3, f=g+h=3$

3 steps to reach GS from here

$g=1, h=4, f=5$

	2	3
1	4	6
7	5	8

$g=1, h=2, f=3$

1	2	3
4		6
7	5	8

$g=1, h=4, f=5$

1	2	3
7	4	6
	5	8

4 steps to reach GS from here as one has to go back to IS & then follow the right path.

$g=2, h=1, f=3$

1	2	3
4	5	6
7		8

$g=2, h=3, f=5$

1	2	3
4	6	
7	5	8

$g=2, h=3, f=5$

1		3
4	2	6
7	5	8

$g=3, h=2, f=5$

1	2	3
4	5	6
	7	8

$g=3, h=0, f=3$

1	2	3
4	5	6
7	8	

GS

Implementation Details:

Each node contains following information.

1. Description of the problem state it represents.
2. How promising it is (HF value)
3. A parent link indicating the node from which it has been expanded (Used to retrace a path once a goal is found).
4. List of nodes generated from it (Successor nodes)

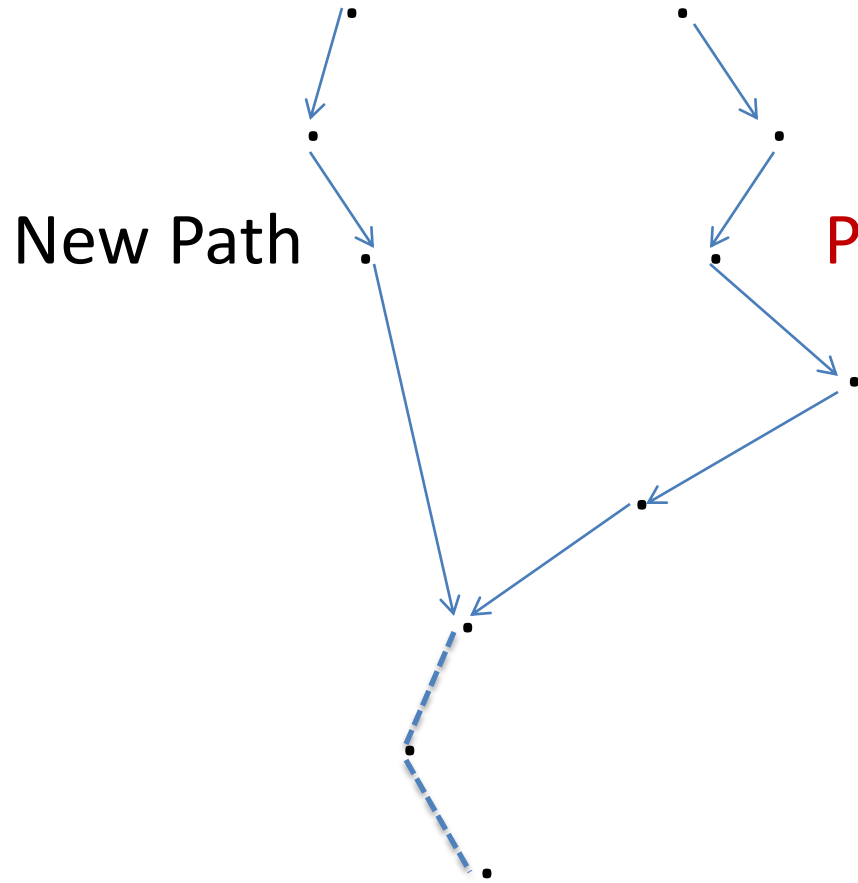
Data structures required for implementation :

OPEN: Contains nodes which have been generated & HF applied on them but not yet expanded (OPEN is a priority queue where the priority is based on HF values).

CLOSED: Contains nodes which have been already expanded (used to ensure that the same node is not examined twice).

Best-First Search (Simplified version of A*)

1. Begin with OPEN containing only the IS node.
2. Loop until a GS is found or OPEN is empty.
 - A) Select the best node from OPEN.
 - B) Add the node to CLOSED.
 - C) Generate its successors.
 - D) for each successor do:
 - i) If it has not yet been generated before then evaluate it by applying the HF & add it to OPEN. Record its parent node.
 - ii) Else change the parent if this new path is better than the previous one, update the cost of arriving at this node and propagate this updated cost to all its successors(if it has been already expanded).

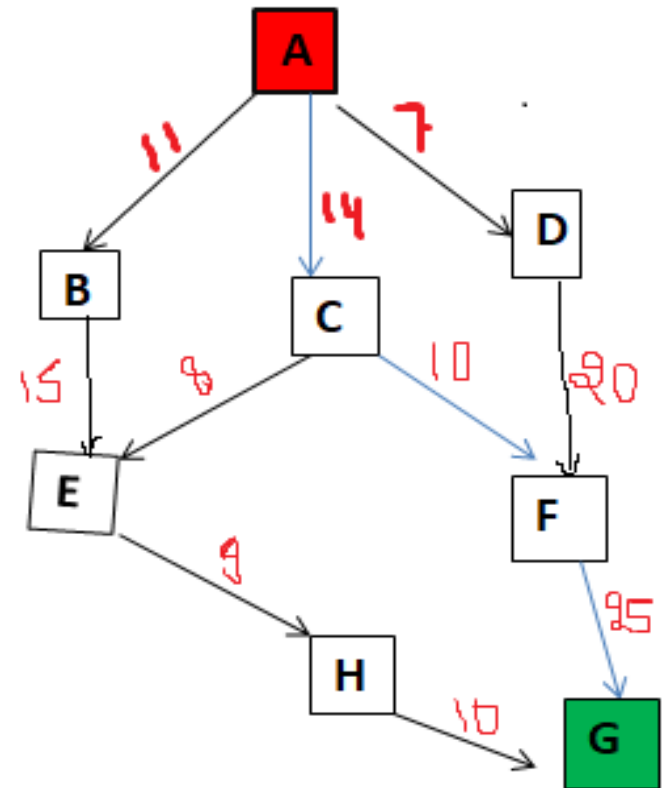


Path: A,D,F=27

Path: A,C,F=24 --> A,C,F,G=24+25=49

Path: A,B,E=26

Path: A,C,E=22 --> A,C,E,H,G=22+9+10=41 (Best path from A to G)



Algorithm A*

1. Insert **IS-node** into **OPEN**

set $g(\text{IS-Node}) = \emptyset$ & $h'(\text{IS-Node}) = \text{estimated value}$
so that $f'(\text{IS-Node}) = (\emptyset + h')$
set **CLOSED** as empty.

2. Loop until GS-Node is found.

i) If **OPEN** is empty, indicate **FAILURE**

Else pick the best node(say lowest f' value) from **OPEN**
and call it **BESTNODE**.

ii) Delete the selected node from **OPEN** & add it to
CLOSED.

iii) IF **BESTNODE** is GS-Node THEN Return solution & Exit.

Else generate successors of **BESTNODE**

For each **successor** Do:

- A) Set a pointer from **successor** to **BESTNODE**.
- B) Compute: $g(\text{successor}) = g(\text{BESTNODE}) +$
cost of moving from **BESTNODE** to **successor**.
- C) If **successor** is already in **OPEN**, then don't insert into **OPEN** but evaluate the new path to **successor**

Determine the parent link:

If the path just found is cheaper than the current
best path then update the parent link & f' value
Else continue with the old values.

D) If SUCCESSOR is not in OPEN but in CLOSED,
Then evaluate the new path & **Determine the Parent link**
If the new path is better, then

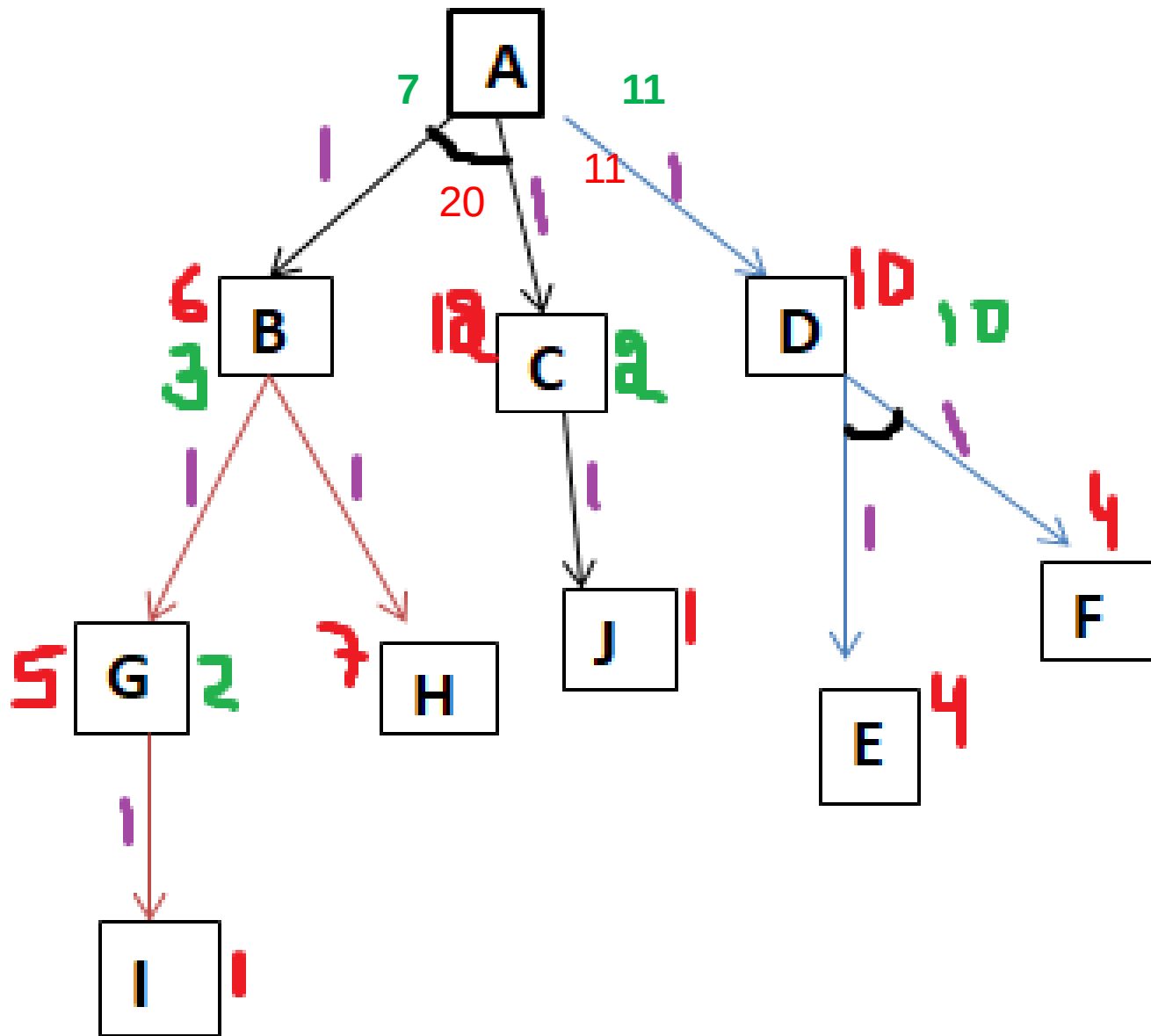
- i) change the parent link
- ii) update g and f' values.
- iii) Propagate the new values of g and f' to the
all the successor nodes below

E) If SUCCESSOR is neither in OPEN nor in CLOSED:
then Add it to OPEN,
Add it to BESTNODE'S SUCCESSOR list,
Compute : $f'(SUCCESSOR) = g(SUCCESSOR) + h'(SUCCESSOR)$

Problem Reduction (And-Or graph)

Suitable for decomposable problems





Algorithm : AO* (And-Or graph)

Let G: Graph representing the state space

S: indicates that a problem at a node is solved

F: Futility limit

1. Begin the process at IS-node.
2. Loop until IS-node is labeled as S or until $\text{cost} > F$ i.e. $f^*(IS) > F$
 - A) Traverse G starting at IS-node by always following the current best path while none of the nodes along the chosen path is left unexpanded (or) labeled as S.

B) Select one of the unexpanded node and expand it (generate its successors)

If it has no SUCCESSOR and it's $h' \neq 0$

then set h' value of this node as F.

(This happens if a sub-problem is unsolvable.)

Else add it's successor to G and compute h' for each of the successors

If h' of any node is $\emptyset$ then label it as S.

// problem at that node is solved

C) Update the change in h' value (if any) to all ancestors along the path **upto the IS-node**.

D) If a successor (in case of OR arc) or successors(in case of AND arc) or labeled as S , then label the **parent node as S** and propagate this backward in order to get the most promising path.